

TRABAJO FINAL

Evolución y configuración controlada de una aplicación

Curso	Evolución y Configuración de Software
Evaluación	Evaluación Final - Trabajo grupal
Puntaje	20 puntos
Fecha de entrega	Semana 7

1. Descripción del trabajo

El trabajo final consiste en seleccionar una aplicación existente y funcional para analizar su estado actual, identificar problemas de mantenibilidad y realizar mejoras de forma controlada. El equipo deberá demostrar cómo la aplicación evoluciona aplicando mantenimiento, refactorización, principios de diseño, gestión de configuración, gestión de cambios y control de versiones con Git.

No se evaluará la cantidad de funcionalidades ni la complejidad de la aplicación. Se evaluará la capacidad del equipo para comprender el software, justificar las mejoras realizadas y demostrar con evidencias cómo se controlaron los cambios y las versiones del proyecto.

2. Indicaciones generales

- El trabajo se desarrolla en equipo y todos los integrantes deben participar en la sustentación.
- Puede utilizarse una aplicación académica o un proyecto desarrollado previamente. No es obligatorio crear un sistema desde cero.
- Las mejoras deben ser reales y estar respaldadas por evidencias del antes y después.
- El repositorio Git debe permitir identificar los cambios realizados y la participación del equipo.
- La documentación debe ser breve, clara y coherente con lo que se muestra en la aplicación y el repositorio.

3. Contenido mínimo del trabajo

3.1 Descripción y diagnóstico inicial

Presentar brevemente la aplicación: propósito, tecnologías, principales módulos o componentes y un diagrama sencillo de su funcionamiento. El equipo deberá identificar al menos dos problemas de mantenibilidad o Code Smells y, si corresponde, alguna evidencia de deuda técnica.

Finalidad: Comprobar que el equipo comprende el software antes de modificarlo y que puede identificar puntos de mejora con criterio técnico.

3.2 Plan breve de mantenimiento

Plantear dos acciones de mantenimiento sobre la aplicación. Para cada una se debe indicar el problema identificado, tipo de mantenimiento, acción propuesta, beneficio esperado, posible riesgo y forma de verificación.

Finalidad: Relacionar un problema real con una acción concreta de mantenimiento y justificar por qué corresponde realizarla.

3.3 Refactorización: antes y después

Seleccionar al menos un componente y mostrar el código o diseño original, el problema detectado y la versión refactorizada. La mejora debe conservar el comportamiento funcional y evidenciar al menos dos conceptos trabajados en el curso: SRP, DRY, OCP, polimorfismo, DIP, IoC o inyección de dependencias.

Finalidad: Demostrar que el equipo puede mejorar la estructura interna del software sin alterar lo que la aplicación debe hacer.

3.4 Gestión de cambios

Documentar dos solicitudes de cambio. Cada una debe incluir código, descripción, motivo, componentes o artefactos afectados, impacto, riesgo y estado. El equipo deberá poder explicar el flujo seguido: solicitud, análisis, aprobación, implementación y pruebas.

Finalidad: Evidenciar que los cambios no se realizan de manera improvisada, sino que son analizados, controlados y documentados.

3.5 Gestión de configuración y baseline

Identificar los principales artefactos controlados en el proyecto, por ejemplo: código fuente, base de datos, scripts, archivos de configuración, documentación o pruebas. Debe definirse una baseline inicial y una baseline final que permitan comparar el estado de la aplicación antes y después de las mejoras.

Finalidad: Reconocer qué elementos deben mantenerse bajo control y cómo se registra el estado del producto.

3.6 Gestión de versiones con Git

Utilizar un repositorio Git y mostrar, como mínimo, historial de commits, una rama de trabajo, integración mediante merge y una versión o tag final. El historial debe ser comprensible y guardar relación con los cambios realizados.

Finalidad: Utilizar el control de versiones como evidencia de la evolución técnica del software, no solo como un medio para almacenar archivos.

3.7 Evidencia de participación del equipo

Debe poder identificarse la participación de todos los integrantes mediante commits, tareas, implementación de cambios, pruebas, documentación u otras evidencias verificables.

Finalidad: Comprobar que el proyecto fue desarrollado de forma colaborativa y que cada integrante conoce el trabajo realizado.

3.8 Uso de Inteligencia Artificial

Indicar brevemente si se utilizó alguna herramienta de IA durante el análisis, mantenimiento, refactorización o pruebas. Si se utilizó, señalar para qué se usó, qué resultado produjo y qué validó o corrigió el equipo. Si no se utilizó, simplemente indicarlo.

Finalidad: Evidenciar un uso responsable de la IA como herramienta de apoyo y no como sustituto del criterio técnico del equipo.

4. Entregables

- Aplicación funcional y repositorio Git del proyecto.
- Informe final en PDF, con un máximo sugerido de 8 páginas sin contar la portada.
- Presentación de 8 a 10 diapositivas para la sustentación.

Estructura sugerida del informe

1. Descripción de la aplicación.
2. Diagnóstico inicial.
3. Plan de mantenimiento.
4. Refactorización: antes y después.
5. Solicitudes de cambio y gestión de configuración.
6. Gestión de versiones y evidencias del repositorio.
7. Participación del equipo y uso de IA.
8. Conclusiones.

Sustentación

Tiempo sugerido: 10 minutos de exposición y 5 minutos de preguntas por equipo. Todos los integrantes deben participar y estar en condiciones de explicar las decisiones técnicas realizadas.

RURICA DEL TRABAJO FINAL

Puntaje Total: 20 Puntos

Criterio	4 puntos – Destacado	3 puntos – Logrado	2 puntos – En proceso	1 punto – En inicio
1. Diagnóstico y plan de mantenimiento	Describe claramente la aplicación, identifica problemas reales de mantenibilidad/deuda técnica y propone 2 acciones de mantenimiento correctamente justificadas.	Presenta un diagnóstico adecuado y acciones coherentes, con pequeñas omisiones.	El diagnóstico o las acciones son parciales y presentan poca justificación.	Presenta un análisis superficial o no relaciona adecuadamente los problemas con el mantenimiento.
2. Refactorización y principios de diseño	Presenta claramente el antes y después, conserva la funcionalidad y aplica correctamente al menos 2 principios, justificando técnicamente la mejora.	La refactorización es adecuada y los principios están correctamente aplicados, aunque existen pequeñas oportunidades de mejora.	La refactorización es parcial o existe confusión en la aplicación de los principios.	No demuestra una refactorización clara o no aplica correctamente los principios estudiados.
3. Gestión de cambios y configuración	Las 2 solicitudes están correctamente documentadas; identifica impacto, riesgos, artefactos y baseline inicial/final, evidenciando trazabilidad.	Gestiona correctamente la mayoría de los elementos solicitados, con omisiones menores.	Presenta solicitudes de cambio, pero el análisis de impacto, riesgos o configuración es limitado.	Los cambios se presentan sin suficiente análisis, control o evidencia.
4. Gestión de versiones y uso de Git	El repositorio evidencia correctamente commits, branch, merge, historial, versión/tag final y una evolución coherente del producto.	Utiliza adecuadamente Git y presenta la mayoría de evidencias requeridas.	Utiliza Git de manera básica, pero existen evidencias incompletas o poca organización del historial.	El repositorio no permite identificar adecuadamente cómo evolucionó el proyecto.
5. Sustentación, evidencias y dominio del proyecto	Presentación clara y organizada; todos participan, explican sus aportes y responden correctamente preguntas técnicas demostrando dominio del trabajo.	Buena presentación y dominio general, con algunas respuestas poco precisas.	La presentación es comprensible, pero existe dominio parcial o participación desigual.	La presentación es incompleta o los integrantes presentan dificultades para explicar y justificar el proyecto.